

Problem Description and project objectives

Problem Description (Application-centered):

- Patients frequently seek medical advice about medications via chatbots, forums, and Q&A websites.
- Some of these questions may imply urgent risks (e.g., incorrect dosage, drug interactions), while others are harmless or general.
- Without proper classification, critical questions may be missed, delaying care or causing harm.

Why It Matters:

- Automatic risk classification helps prioritize urgent questions and reduce delays in medical consultation.
- Enhances **patient safety** and improves chatbot reliability in real-world clinical settings.

Why It's Challenging:

- Questions are often short, vague, and written in layman's language.
- High variability in phrasing and noisy inputs make classification hard.
- Lack of large labeled medical QA datasets makes training robust models difficult.

Project Objectives

- Explore and compare multiple LLM-based approaches to classify risk level from medication-related questions.
- Evaluate whether retrieval-based context (RAG) improves LLM performance.
- Investigate the impact of domain-specific pretraining (e.g., **BioBERT**, **BlueBERT**) vs. general LLMs (**GPT-4.1**).

Dvora Goncharok
&
Arbel Shifman



Formal Task Specification

Task Overview:

- Objective: Classify free-text patient questions about medications by their clinical risk level: General (safe) vs. Critical (requires medical attention).
- Motivation: Automate risk triage in clinical chatbots using LLM-based methods.

Input and Output:

- Input:
 - A single natural-language question (English), written by a patient.
 - (In RAG setup): Up to k retrieved context passages from a medical knowledge corpus.
- Output:
 - A binary label: General or Critical.

Evaluation Metrics:

- Metrics: Accuracy, Precision, Recall, F1-score (per class)
- Additional: Confusion Matrix, class-wise analysis, overfitting detection (Train vs. Test)



Project Plan (High-Level Steps):

1. Data Preparation & EDA

- Load and clean MedInfo2019-QA-Medications dataset
- Manual labeling (General vs. Critical) and class-imbalance analysis

2. Baseline & Feature Engineering

- TF-IDF + SMOTE for text features and class balancing
- Critical Similarity feature (cosine similarity to known critical examples)

3. Baseline Models (trained on TF-IDF + Critical Similarity):

- Logistic Regression
- Linear SVM
- Random Forest
- K-Nearest Neighbors (KNN)
- Gradient Boosting
- Evaluate each on the validation split (Accuracy, Precision, Recall, F1)

4. Biomedical LLM Fine-Tuning

- Use RAG: build FAISS index over medical corpus
- For each test question: retrieve top-3 passages
- Fine-tune BioBERT and BlueBERT on the training split (80/20)
- Evaluate on test set (Accuracy, Precision, Recall, F1)

5. GPT-4.1: Classification Only

- Use RAG: build FAISS index over medical corpus
- For each test question: retrieve top-3 passages
- Prompt GPT-4.1 to classify the question as "General" or "Critical"
- Evaluate performance

6. GPT-4.1: Generate + Classify

- Use RAG: build FAISS index over medical corpus
- For each test question: retrieve top-3 passages
- Prompt GPT-4.1 to generate additional Critical questions
- Augment the training data with synthetic samples
- Re-run classification using the RAG + prompt setup
- Measure performance improvements, especially for Critical recall

7. Final Comparison & Analysis

1. Compare all models: BioBERT, BlueBERT, GPT-4.1 (both variants) vs. baselines
2. All models use RAG-augmented retrieval for fair comparison
3. Report metrics per class.

Prior Art

Source / Title	Task solved	Approach / Model	Data	Metrics	Results
Risk of mortality and cardiopulmonary arrest in critical patients presenting to the emergency department using machine learning and natural language processing	Predictive modeling task: Estimate the risk of in-hospital mortality or cardiopulmonary arrest within 24 hours of emergency department (ED) triage using machine learning and natural language processing techniques.	XGBoost	40,218 emergency department (ED) patient questions	AUROC = 0.96	High accuracy in predicting mortality and cardiac arrest within 24 hours
Identifying the Perceived Severity of Patient-Generated Telemedical Queries Regarding COVID: Developing and Evaluating a Transfer Learning-Based Solution	Classification task: Automatically classify patient-generated COVID-related queries by perceived severity level (mild, medium, severe) using transfer learning.	SBERT contextual embeddings	11,746 telemedicine queries from eConsult platform	F1 score = 0.917	Effective at classifying severe vs. non-severe queries
COMPARISON OF PERFORMANCES OF OPEN ACCESS NATURAL LANGUAGE PROCESSING BASED CHATBOT APPLICATIONS IN TRIAGE DECISIONS	Evaluation task: Compare the accuracy of different open-access NLP-based chatbot applications in making triage decisions.	GPT-4	130,974 high-acuity patient queries categorized as ESI-1 or ESI-2 (Emergency Severity Index)	F1 score = 0.899	High agreement with emergency medicine experts
Comparison of Diagnostic and Triage Accuracy of Ada Health and WebMD Symptom Checkers, ChatGPT, and Physicians for Patients in an Emergency Department: Clinical Data Analysis Study	Evaluation task: Assess and compare the diagnostic and triage accuracy of AI symptom checkers (Ada, WebMD, ChatGPT) and emergency physicians using real clinical cases.	ChatGPT 3.5 & 4.0, Ada, WebMD	40 real patient cases from an emergency department	Top-1 Match: ChatGPT 4.0: 33% Physicians: 47%	ChatGPT models underperformed in diagnostic accuracy compared to physicians

Prior Art

Source / Title	Task solved	Approach / Model	Data	Metrics	Results
Performance of emergency triage prediction of an open access natural language processing based chatbot application (ChatGPT): A preliminary, scenario-based cross-sectional study	Evaluation task: Evaluate the emergency triage prediction accuracy of ChatGPT based on standardized clinical case scenarios.	ChatGPT	50 simulated ED patient scenarios	F1 score = 0.461 Cohen's kappa = 0.341	Moderate agreement with specialists; potential for improvement
Human intelligence versus Chat-GPT: who performs better in correctly classifying patients in triage?	Comparative evaluation task: Compare the accuracy of ChatGPT versus human physicians in classifying patient triage levels based on clinical scenarios.	ChatGPT	30 simulated triage case vignettes	Cohen's kappa = 0.278	Triage nurses consistently outperformed ChatGPT

Data Preparation/Description

Pipeline Overview:

- **Input:** Free-text medication-related question (e.g., about dosage, side effects, interactions)
- **Output:** Risk level classification – General (safe) or Critical (dangerous)
- **Task Type:** Binary class text classification problem using an NLP pipeline

Labeling Process (Risk-Level)

- New Field: Risk_Level
- General: Non-urgent or informational queries (e.g., “Can I take ibuprofen with grapefruit?”)
- Critical: Potentially life-threatening issues (e.g., “Is it safe to combine warfarin and aspirin?”)
- Workflow:
 1. First pass: One annotator labeled each question.
 2. Verification: Second team member reviewed using established guidelines.
 3. Class Imbalance: ~20% of questions labeled “Critical,” 80% “General”

Source Dataset:

- **Name:** MedInfo2019-QA-Medications (publicly available on GitHub) [Link to the Data Set](#)
- **Description:** A dataset of patient-generated questions about medications, originally collected from public medical forums and Q&A platforms.

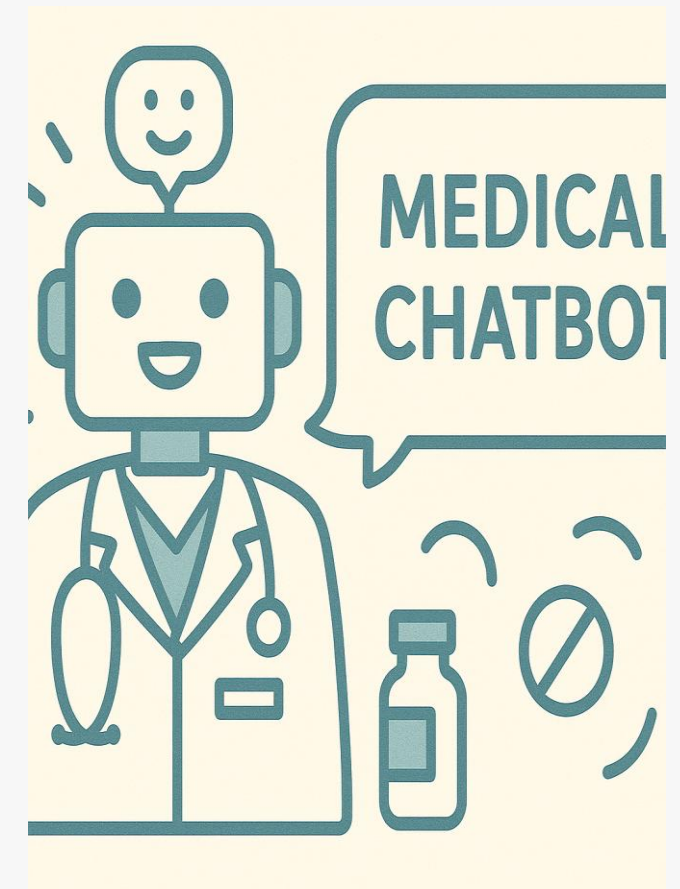


Dataset Properties (EDA):

- Total Samples: 652 (after removing duplicates/empty entries)
- Class Distribution:
 - General: 522 questions (80%)
 - Critical: 130 questions (20%)
- Question Length (words):
 - Mean: 12 ± 6 words
 - Range: 3–35 words
- Text Characteristics:
 - Mostly short, colloquial phrasing
 - Common keywords in “Critical” class: “overdose,” “interaction,” “allergic”

Synthetic Data Generation

- Goal: Address scarcity of truly “Critical” examples
- Method:
- Before fine-tuning, we applied SMOTE to the TF-IDF-derived embeddings to generate additional “Critical” examples. These synthetically balanced embeddings were then concatenated with the original training set.
- BERT-Based Model Balancing:
 - Oversampling strategy so that each mini-batch contained roughly equal numbers of General vs. Critical questions.
 - Class weights into the cross-entropy to penalize mistakes on the minority (Critical) class more heavily.



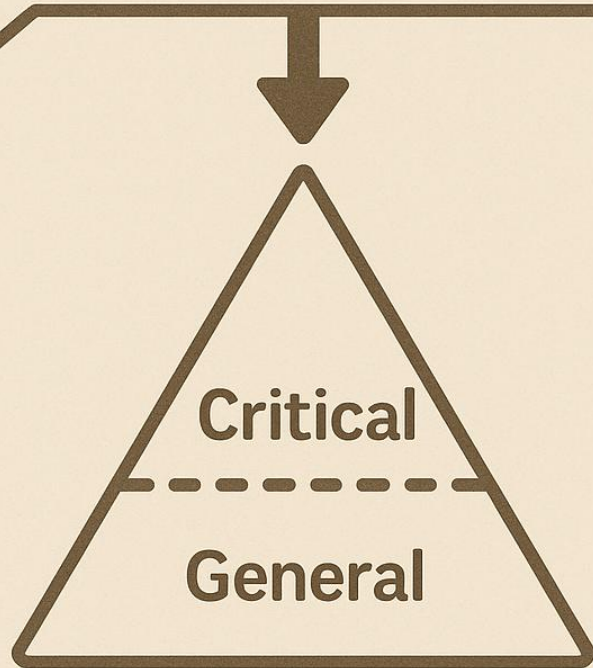
- Prompt GPT-4.1: “Generate 50 medication-related questions that clearly indicate urgent or dangerous scenarios (overdose, severe interactions, allergic emergencies).”
- Review & filter output for relevance; assign label “Critical”
- Augmented Training Set: Combine original questions + GPT-4.1-generated critical questions

Examples:

Question_Text	Risk_Level
"Is it safe to take ibuprofen while on blood thinners?"	Critical
"Can I split my metformin pill to reduce dose?"	General
"What happens if I accidentally take 50mg of lisinopril twice?"	Critical
"Is it okay to drink grapefruit juice with my medication?"	General

Risk classification

- Is it safe to take ibuprofen while on blood thinners?
- Can I split my metformin pill to reduce dose?
- What happens if I accidentally take 50mg of lisinopril twice?

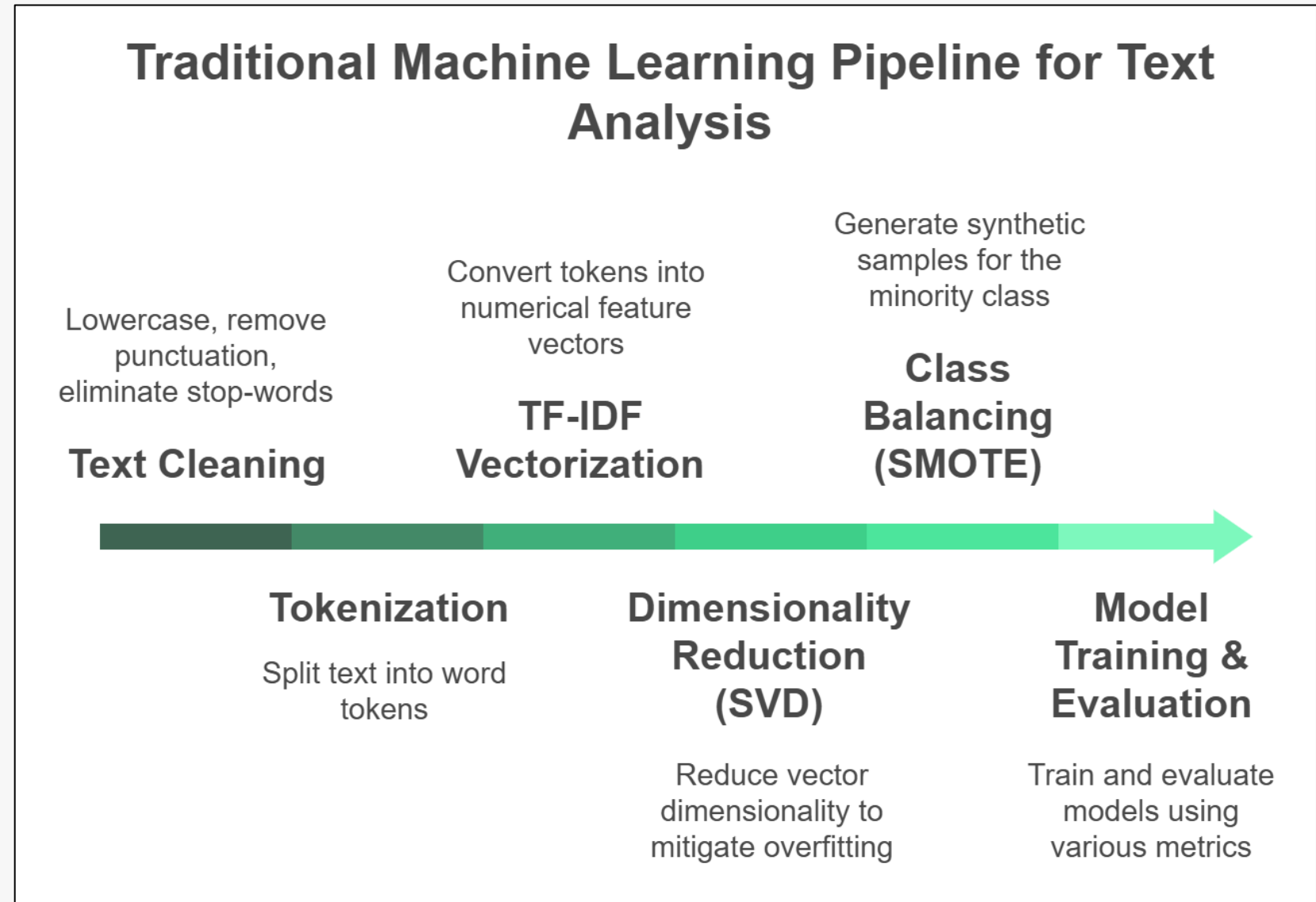


**Binary
classification**

Models and processing pipelines

Models Used:

- Traditional ML models:
 - Logistic Regression, Random Forest, SVM, Gradient Boosting, KNN, SGD (L2)



Advanced LLMs:

- **RAG (Retrieval-Augmented Generation):**
 1. Build retrieval index (FAISS) over training questions (embeddings via DPRContextEncoder)
 2. Retrieve top-k similar Q&A contexts (using DPRQuestionEncoder + FAISS search)
 3. RAG was integrated across all advanced models to enrich their training data. Using a FAISS-based retrieval system, we provided each model with relevant examples, thus improving their ability to classify questions accurately by bringing the most similar and informative contexts closer to the model.
- 1. **BioBERT Fine-Tuning**
 - Base model: dmis-lab/biobert-base-cased-v1.1
 - Task head: Add a binary classification layer (risk vs. safe) on top of BioBERT's pooled output.
 - Pipeline:
 2. Tokenize questions with BioBERT tokenizer.
 3. Fine-tune on the training split (80/20):
 - Optimizer: AdamW, LR=2e-5, epochs=3, batch_size=16
 - Loss: Cross-entropy (with class weights to mitigate imbalance)
 4. Evaluate on test set (Accuracy, Precision, Recall, F1).
- 2. **BlueBERT Fine-Tuning**
 - Base model: bionlp/bluebert_pubmed_mimic_uncased_L-24_H-1024_A-16 (or equivalent BlueBERT checkpoint)
 - Task head: Same binary classification head as BioBERT
 - Pipeline:
 2. Tokenize with BlueBERT.
 3. Fine-tune using identical hyperparameters (AdamW, LR=2e-5, epochs=3, batch_size=16).
 4. Evaluate on test set (Accuracy, Precision, Recall, F1).

3. GPT-4.1 Classification Only Model: GPT-4.1 via Azure/OpenAI API

- Retrieval Setup: DPR + FAISS
 1. Index Construction
 - Encode all training questions using DPRContextEncoder → 768-dim vectors
 - Build FAISS index over embeddings
 2. Query Retrieval
 - Encode each test question with DPRQuestionEncoder
 - Retrieve top-3 most similar contexts from FAISS
 3. Classification
 - Construct a prompt:
“Below is a patient medication question plus relevant context. Classify as 'General' or 'Critical'.”
 - Send prompt to GPT-4.1 and parse response
 - Compute test metrics

4. GPT-4.1 Generate + Classify

- Goal: Enrich rare Critical examples via generation
- Pipeline:
 1. Prompt GPT-4.1 to generate new Critical questions
 - Use structured prompts with examples
 - Filter out non-medical/invalid outputs
 2. Augment the training set with generated Critical examples
 3. Re-run classification using the same RAG-based prompt structure as above and compute test metrics

Advanced LLM Training and Classification Process

Build Retrieval Index

Create a FAISS index over training questions using DPRContextEncoder.

Classify with GPT-4.1

Classify questions using GPT-4.1 with retrieved passages in the prompt.

Fine-Tune BlueBERT

Fine-tune BlueBERT with a binary classification layer and evaluate performance.

GPT-4.1 Generate + Classify

Generate new Critical questions and augment the training set.



Retrieve Similar Contexts

Use DPRQuestionEncoder and FAISS search to find top-k similar contexts.

Fine-Tune BioBERT

Fine-tune BioBERT with a binary classification layer and evaluate performance.

GPT-4.1 Classification Only

Classify questions using GPT-4.1 with DPR and FAISS retrieval.

Training Configuration

- Data Split: Stratified 80 % train / 20 % test
- Cross-Validation: 5-fold (for traditional ML models)
- Platform: Google Colab (Python 3.8), Tesla T4 GPU
- Libraries: PyTorch, HuggingFace Transformers, scikit-learn, faiss-cpu, imbalanced-learn (SMOTE)

Model	Epochs	Batch Size	Learning Rate	Weight Decay
BioBERT	3	16	2×10^{-5}	0.01
BlueBERT	3	16	2×10^{-5}	0.01
GPT-4.1 (Classify only)	-	-	-	-
GPT-4.1 (Generate + Classify)	-	-	-	-

Metrics

1. Text-Similarity Feature

- Cosine similarity on TF-IDF vectors → Critical-Similarity score

2. Classification Metrics

- Accuracy = $(TP + TN) / (P + N)$
- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1-score = $2 \cdot (Precision \cdot Recall) / (Precision + Recall)$
- Macro-F1 (average F1 over classes) to mitigate class-imbalance

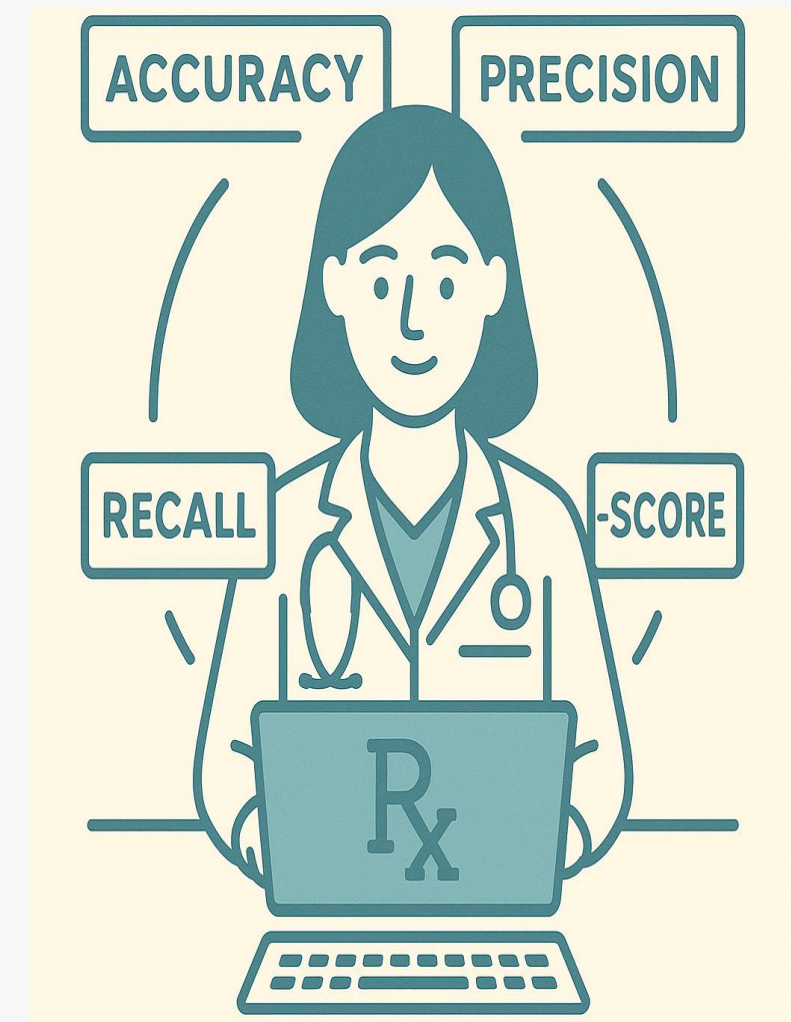
3. Retrieval Metric (RAG only)

- Recall@5 % of test questions for which at least one truly relevant context appears in the top-5 FAISS results.

4. Data Splits & Protocol

- Dataset:
652 questions → Stratified 80% train / 20% test
- Traditional ML (TF-IDF + Critical-Similarity + SMOTE):
 - 5-fold CV on 80% training split
 - Final evaluation on held-out 20% test set
- LLM Fine-Tuning (BioBERT, BlueBERT):
 - From 80% train, hold out 20% for validation
 - Track metrics per epoch: Accuracy, Precision, Recall, Macro-F1, Loss
 - Early stopping after 2 epochs without Macro-F1 improvement
 - Final test on held-out 20%

- GPT-4.1 – Classification Only (RAG-based):
 - No fine-tuning (inference only)
 - Each test question → prompt with top-3 retrieved passages from FAISS
 - Evaluate predictions on 20% test set
- GPT-4.1 – Generate + Classify:
 - Generate new Critical questions using GPT-4.1 prompt
 - Augment training data
 - Re-run classification as above
 - Compare gains in Macro-F1 and Critical recall



Implementation Details

- Traditional ML & CV:
scikit-learn (LogisticRegression, SVC, RandomForestClassifier),
cross_val_score for 5-fold evaluation
- TF-IDF & SMOTE:
TfidfVectorizer from scikit-learn and SMOTE from imbalanced-learn
- DPR + FAISS (RAG):
HuggingFace's transformers for DPRContextEncoder /
DPRQuestionEncoder, faiss-cpu for indexing and retrieval (IVFFlat, k=3)
- LLM Fine-Tuning (BioBERT, BlueBERT): HuggingFace Trainer API + evaluate
callbacks (compute metrics at each epoch)
- GPT-4.1 (Inference via RAG):
Azure OpenAI SDK (gpt-4-1106-preview)
 - used with structured prompts, no training/fine-tuning
 - applied in both classification and generate + classify modes



Code Organization

- **GitHub Repository:** <https://github.com/Dvora-coder/LLM-Medication-QA-Risk-Classfier-MediGuard>
- **Data Files:**
- [MedInfo2019-QA-Medications-df](#)
 - Raw QA dataset; fields: Question, Focus (Drug) , Question Type, Answer, Section Title, URL.
- [labeled data set](#)
 - Labeled data set with the new column: Risk_Level
- [DDI_data \(drugbank\)](#)
 - A table of drug-drug interaction records, with columns:
 - drug1_id, drug2_id — unique identifiers (e.g., DrugBank IDs)
 - drug1_name, drug2_name — drug names
 - interaction_type — description of the interaction (e.g., “serum concentration,” “risk or severity of bleeding”)
- [EML_export \(WHO Model List of Essential Medicines\)](#)
 - An export of the Essential Medicines List, with columns:
 - Medicine name — the official name of each medicine
 - Indication — approved clinical use or condition (e.g., “Migraine,” “Acute myocardial infarction”)
- [knowledge_corpus](#)

A combined knowledge base of drug information from two sources:

 - DDI interactions (columns: drug1, drug2, interaction)
 - EML indications (columns: medicine_name, indication)

Each row also has:

 - source (either “DDI” or “EML”)
 - text (a natural-language sentence)
- [overlapping_entries.xlsx](#)
 - A filtered subset of knowledge_corpus containing only those entries whose drug appears in both DDI and EML sources.
 - Columns are identical to the full corpus (drug1, drug2, interaction, medicine_name, indication, source, text), but limited to overlapping drug names.

Notebooks & Scripts:

- EDA & Preprocessing ([notebooks/01_EDA_Preprocessing.ipynb](#))
- Baseline Training Data cleaning, tokenization, TF-IDF, SVD, Critical-Similarity feature, TF-IDF + SMOTE → Logistic Regression, SVM, Random Forest, etc. → CV & evaluation ([baseline training](#))
- LLM Fine-Tuning:
 - BlueBERT ([BlueBERT](#)); BioBERT ([BioBERT](#)):
 - Loads augmented_train.pkl, tokenizes with BioBERT/BlueBERT tokenizer
 - Fine-tunes models (AdamW, LR=2e-5, epochs=3, batch_size=16)
 - Tracks validation metrics, saves best checkpoints to /models/bio/ and /models/blue/
 - GPT 4.1 ([GPT4.1](#)):
 - Reads hold-out test questions from labeled_data
 - For classification only: constructs RAG-based prompts with top-3 retrieved passages
 - For generation: prompts GPT-4.1 to create new Critical examples

Results Files:

- Per-Model CSVs (created by each notebook)
 - <model_name>_metrics.csv (e.g., biobert_metrics.csv, bluebert_metrics.csv, gpt4_metrics.csv, rag_metrics.csv)
- Columns:
 - question_id – Unique question identifier
 - true_label – Ground-truth class (General or Critical)
 - predicted_label – Model's predicted class
 - precision_general – Precision for the "General" class
 - precision_critical – Precision for the "Critical" class
 - recall_general – Recall for the "General" class
 - recall_critical – Recall for the "Critical" class
 - f1_general – F1-score for the "General" class
 - f1_critical – F1-score for the "Critical" class
 - macro_f1 – (Precision+Recall) averaged across classes

Each notebook (BioBERT, BlueBERT, GPT-4.1, RAG) automatically exports its own metrics CSV and comparison plots so that all final results are available in the /results/ directory.

For example, the results of the model GPT 4.1 (generate and classify)

```
===== CLASSIFICATION REPORT (both classes) =====
              precision    recall  f1-score   support

   General      0.8908      0.9725      0.9298        109
   Critical      0.9000      0.6750      0.7714         40

 accuracy              0.8926        149
 macro avg      0.8954      0.8237      0.8506        149
 weighted avg      0.8932      0.8926      0.8873        149

===== AVERAGE METRICS =====
Macro Avg – Precision: 0.8954, Recall: 0.8237, F1: 0.8506
Weighted Avg – Precision: 0.8932, Recall: 0.8926, F1: 0.8873
```

Intermediate/Baseline:

Baseline Models (TF-IDF + Critical-Similarity + SMOTE):

1. Linear SVM (best baseline):
 - Test Accuracy: 0.77
 - Test F1-score (macro): 0.77
 - Precision (Critical): 0.75
 - Recall (Critical): 0.79
2. Logistic Regression
 - Test Accuracy: 0.76
 - Test F1-score (macro): 0.76
 - Precision (Critical): 0.73
 - Recall (Critical): 0.78
3. Gradient Boosting
 - Test Accuracy: 0.75
 - Test F1-score (macro): 0.75
 - Precision (Critical): 0.72
 - Recall (Critical): 0.76
4. Random Forest
 - Test Accuracy: 0.73
 - Test F1-score (macro): 0.72
 - Precision (Critical): 0.70
 - Recall (Critical): 0.72

5. KNN

- Test Accuracy: 0.50
- Test F1-score (macro): 0.10
- (Very low recall/precision on “Critical” → $F1 \approx 0.10$)

Note: All baselines use the same feature set (TF-IDF + Critical-Similarity) with SMOTE oversampling. SVM is the strongest, achieving ~77% accuracy and F1. KNN fails on the rare “Critical” class.

Intermediate Feature: Critical-Similarity Distribution

- Distribution on Test:
 - 75% of true “Critical” questions have a cosine-similarity ≥ 0.35
 - Only 20% of “General” questions exceed that threshold
- Simple Threshold Rule (score $\geq 0.35 \rightarrow$ “Critical”)
 - Recall (Critical): 0.35 (i.e., only 35% of actual Critical are captured)
 - Shows that the similarity feature alone has limited standalone power.

What We Compare Against:

- Our advanced LLM approaches (BioBERT, BlueBERT, GPT-4.1) will be evaluated on the same 20% test split, so we can directly measure gains over these ~0.77 F1 baselines—particularly on the under-represented “Critical” class.

Main Results

All metrics computed on the same 20 % test split for fair comparison.
BioBERT/BlueBERT results come from fine-tuning on train+validation; GPT-4.1 variants use prompt-based inference.

Model	Accuracy	Macro-F1	Macro-Recall	Macro-Precision
BioBERT (fine-tuned)	0.92	0.90	0.91	0.90
BlueBERT (fine-tuned)	0.9180	0.90	0.90	0.90
GPT-4.1 (Classify)	0.8702	0.7950	0.7950	0.7796
GPT-4.1 (Generate + Classify)	0.8926	0.8506	0.8237	0.8954

GPT 4.1 model (only classify):

- Use a Retrieval-Augmented Generation (RAG) pipeline to fetch relevant context passages.
- Leverage prompt engineering with GPT-4.1 on Azure for high-accuracy classification.

Instructions:

You are a medical classification assistant **with expertise in evaluating drug-drug interactions, adverse effects, and clinical safety.**

Always **prioritize** identifying potential life-threatening interactions, even if they are **rare**.

Given a question about drug interactions or medical usage, classify it as "General" (no immediate risk) or "Critical" (potentially dangerous interaction).

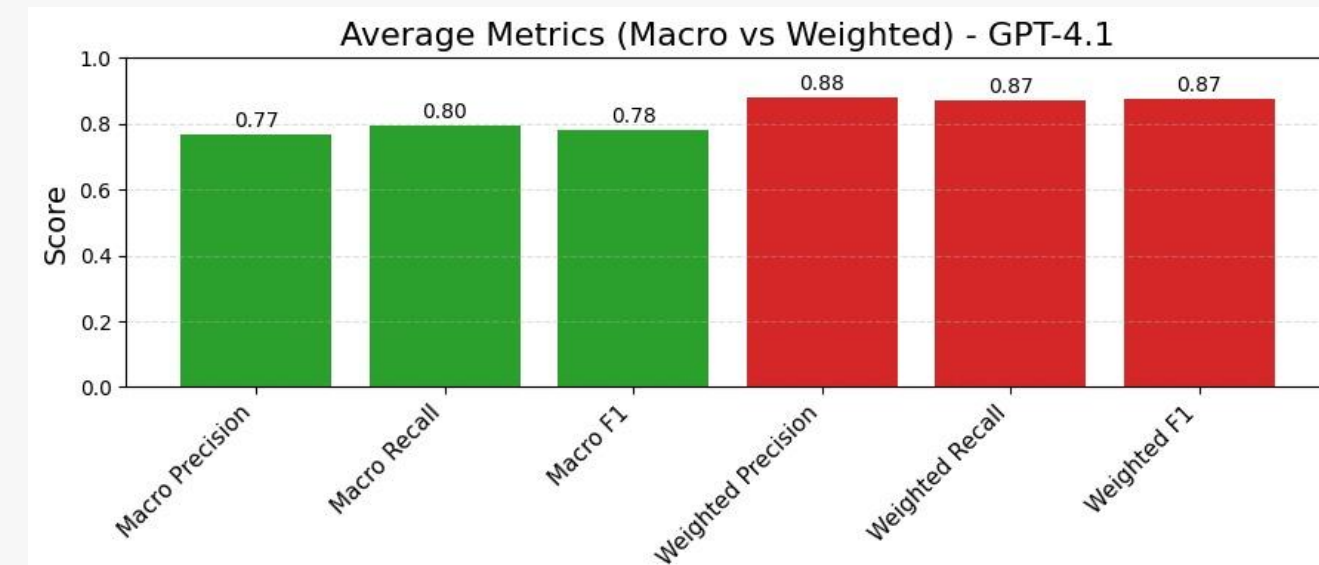
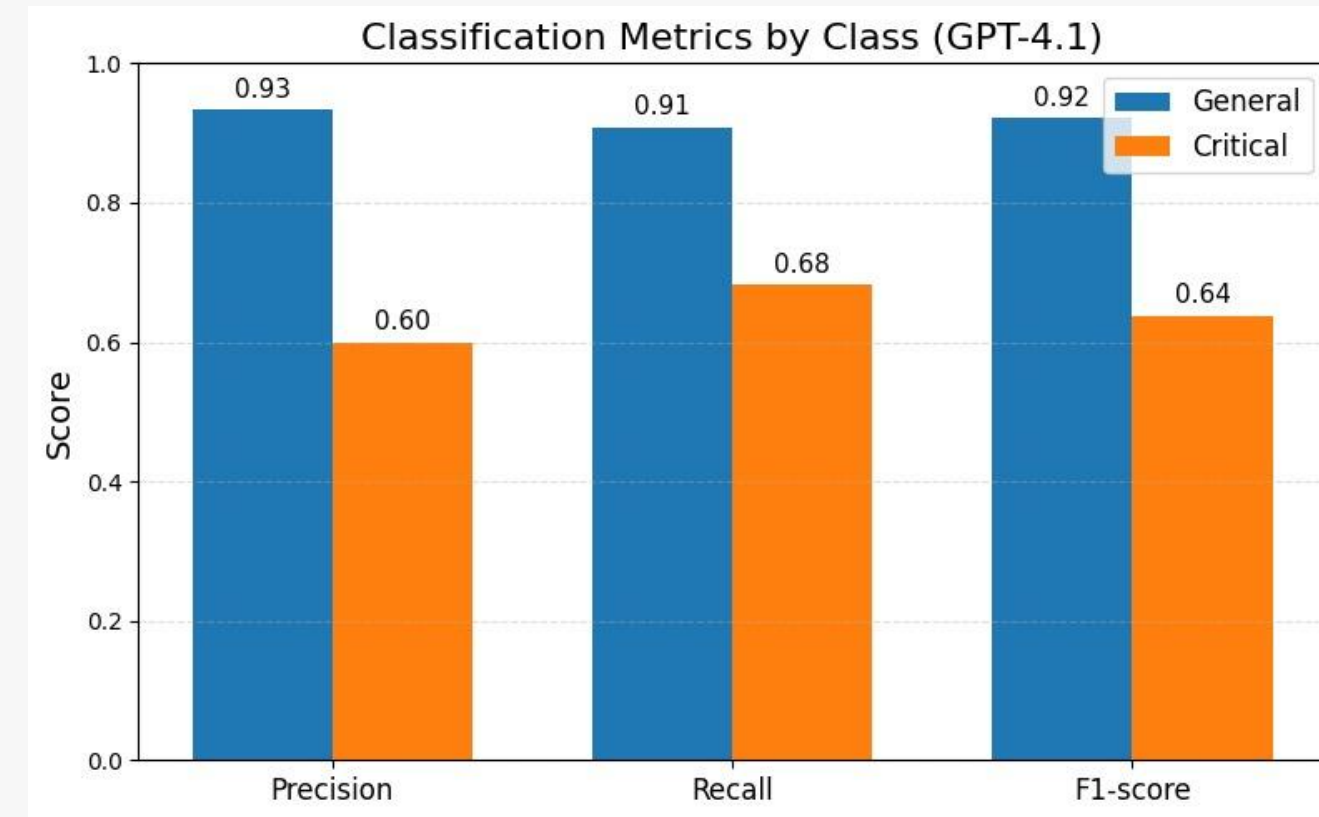
Always use the "Relevant context" passages provided below to inform your classification decision.

Do not provide explanations—answer only with the single word "General" or "Critical".

Examples:

1) Question: "Does taking warfarin with aspirin increase bleeding risk?" -> "Critical"

2) Question: "What is the standard adult dose of amoxicillin for a mild ear infection?" -> "General"



GPT 4.1 model (generate and classify):

Synthetic Critical-Question Generation Prompt:

You are a medical expert specializing in pharmacology and drug-drug interaction safety.

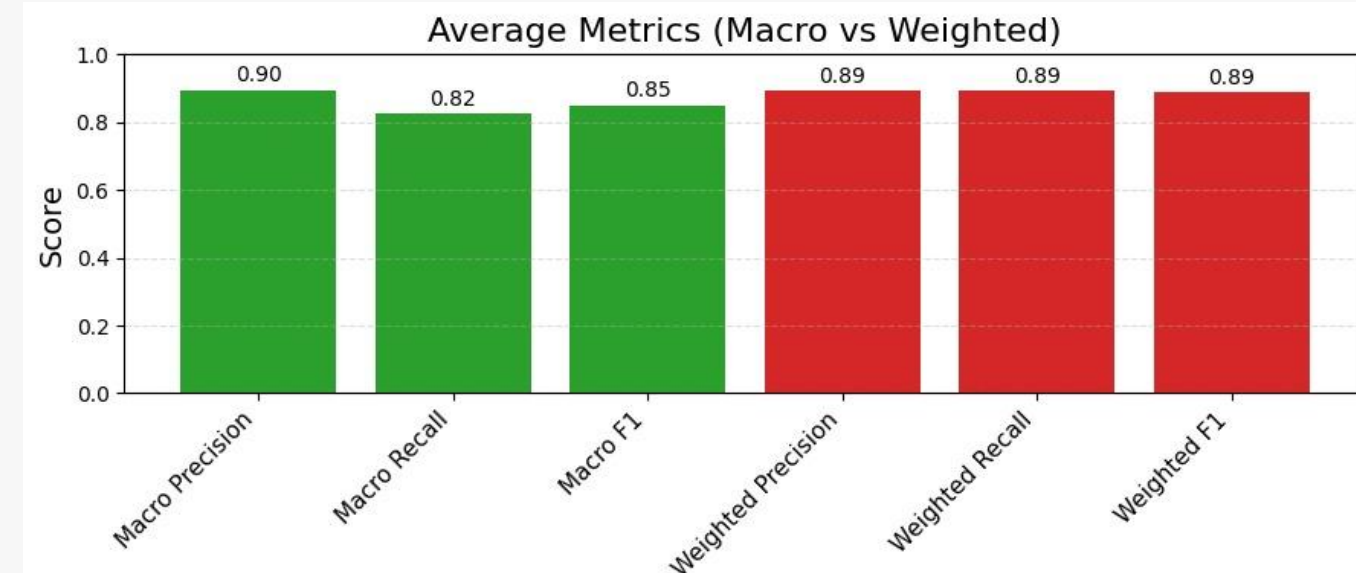
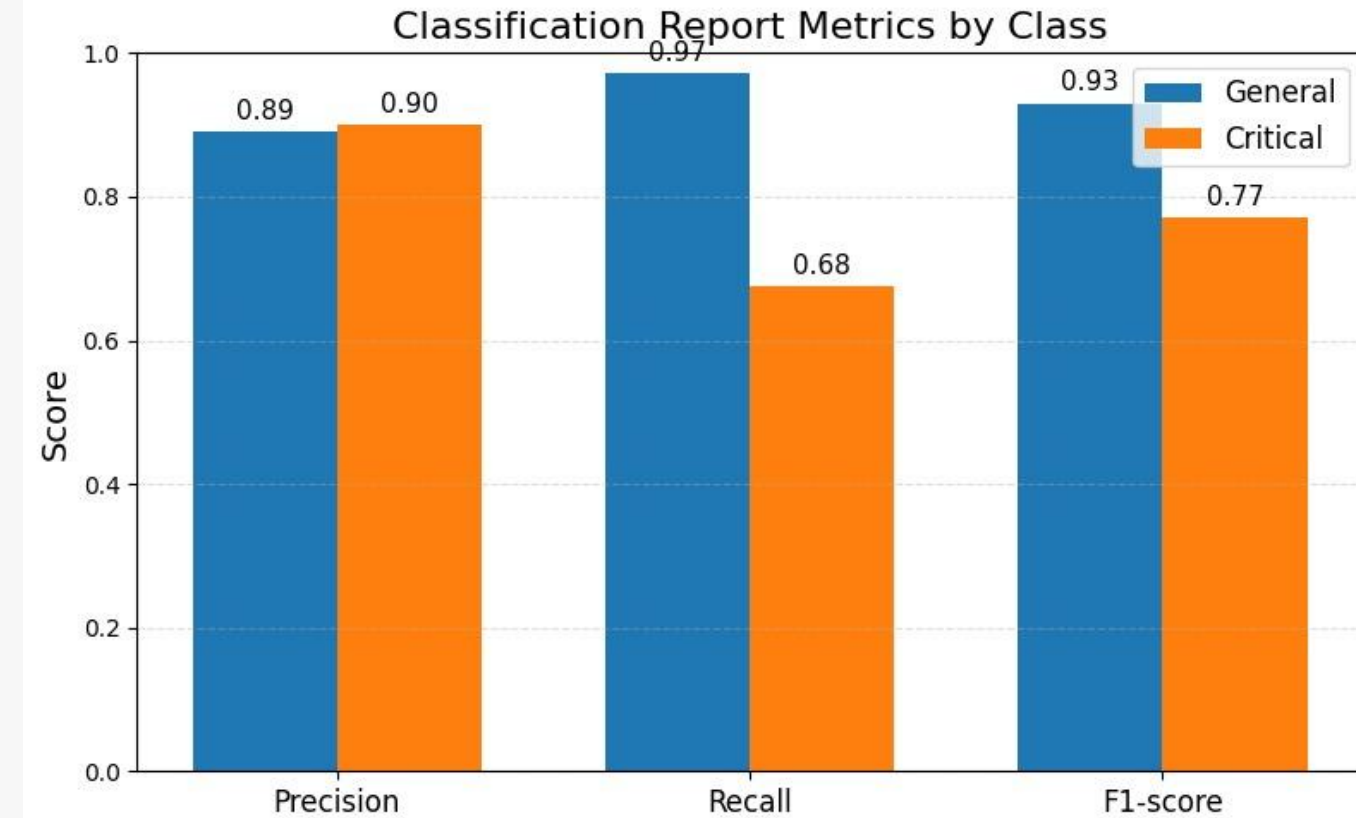
Generate 1 new, unique medical question about a drug interaction that should be classified as "Critical"

(i.e., a combination with potential life-threatening adverse effects), based on this example: ...

- Use only real, commonly prescribed drug names.
- Ensure the question involves two or more drugs known or plausibly capable of dangerous interactions.
- Vary the therapeutic classes (e.g., anticoagulants, antibiotics, antiarrhythmics, antidepressants, statins).
- **Do not invent fictitious drug names or unusual indications.**
- Format the question as a single line, without numbering or quotes.

Example:

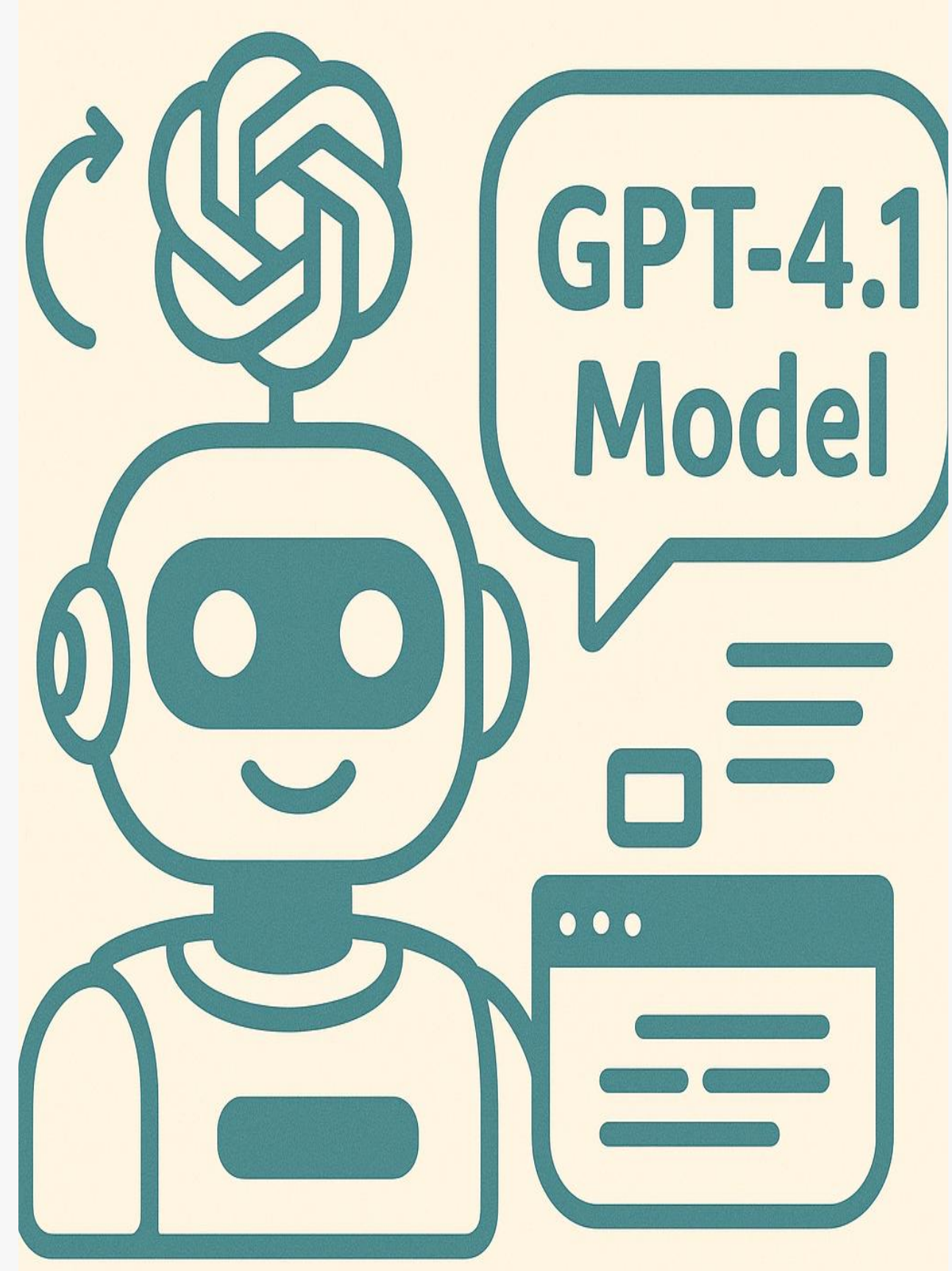
Does combining warfarin with aspirin increase bleeding risk?



GPT 4.1 model (generate and classify):

5 synthetic Critical questions:

1. Is it safe to take fluoxetine with linezolid due to the risk of serotonin syndrome?
2. Is it dangerous to take amiodarone and simvastatin together due to risk of severe muscle breakdown?
3. What are the dangers of taking fluconazole together with simvastatin?
4. Can coadministration of amiodarone and simvastatin increase the risk of severe muscle toxicity?
5. What are the dangers of co-administering amiodarone and simvastatin in patients with cardiac arrhythmias?



Intermediate Results & Baseline Comparison

Did We Achieve Our Objectives?

- Objective: Automatically flag “Critical” vs. “General” medication questions for faster triage.
- Key Finding: **All transformer-based approaches outperformed classical ML.**

Conclusion:

The best-performing model was **BioBERT (fine-tuned)**, **slightly outperforming BlueBERT and all GPT-4.1 variants.**

All LLM-based models (BioBERT, BlueBERT, GPT-4.1) significantly outperformed the traditional baselines.

GPT-4.1 with Generate + Classify achieved higher recall on the “Critical” class, suggesting that augmenting the context with synthetic questions helped improve detection of rare cases — beyond what RAG alone provided.

BioBERT outperformed all models because it was pre-trained specifically on biomedical corpora, making it more sensitive to the structure and terminology of real-world medication-related questions.

Intermediate Results & Baseline Comparison

BioBERT (fine-tuned):

- Based on a BERT model **pre-trained on biomedical literature** (PubMed abstracts and PMC full-texts), then fine-tuned on our labeled dataset.

Best performance due to domain-specific pretraining on medical texts, closely aligned with the task.

BlueBERT (fine-tuned):

- Built on BERT with **combined training on clinical notes (MIMIC-III) and PubMed articles.**

Strong performance, but slightly less aligned than BioBERT with medication-related QA phrasing.

GPT-4.1 (Classify):

- General-purpose LLM accessed **via prompt engineering** and RAG context from our drug-related corpus, no task-specific fine-tuning.

Flexible and powerful, but less specialized in medical terminology.

GPT-4.1 (Generate + Classify):

- Same setup as above, with added synthetic Critical examples to enrich rare-case coverage.

Improved Critical recall, though still limited by lack of full fine-tuning on biomedical domain.

Visual Abstract:

